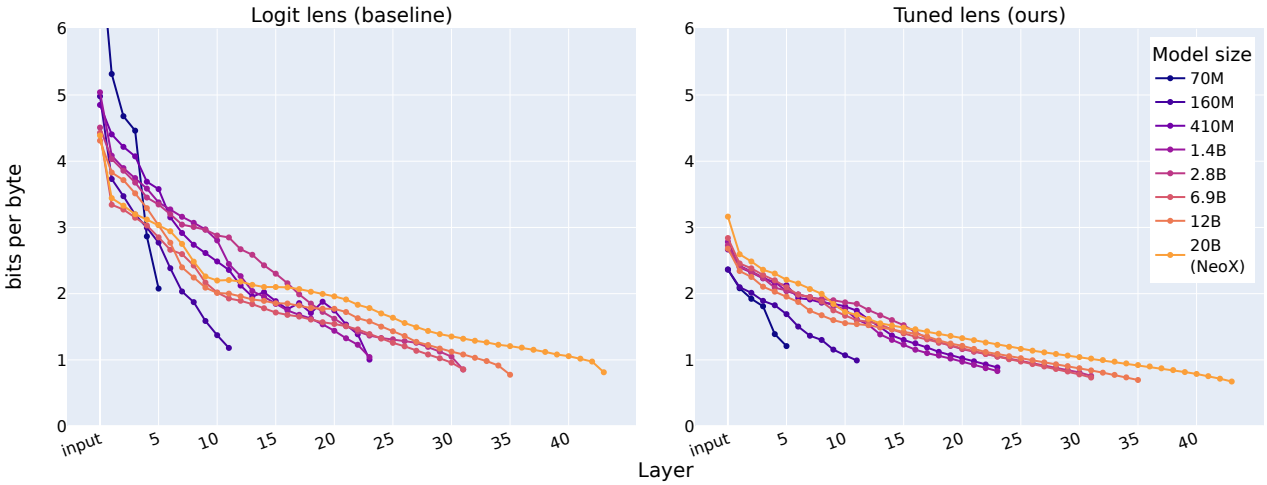

Figure 5. Perplexity of latent predictions elicited by the logit lens (left) and the tuned lens (right) from Pythia and GPT-NeoX-20B, as a function of layer index and model size. Tuned lens predictions are uniformly lower perplexity and exhibit lower variance across independently trained models.

a replacement value is somewhat arbitrary– the network might learn to rely on $\sum_{\ell'=\ell}^{L} \mathbb{E}[F_\ell(\boldsymbol{h}_\ell)]$ as a bias term.

Our first change to the method is to replace the summed residuals with a learnable constant value $\mathbf{b}_\ell$ instead of zero:

$$\text{LogitLens}_\ell^{\text{debiased}}(\boldsymbol{h}_\ell) = \text{LogitLens}(\boldsymbol{h}_\ell + \mathbf{b}_\ell) \quad (7)$$

Representation drift. Another issue with the logit lens is that transformer hidden states often contain a small number of very high variance dimensions, and these "rogue dimensions" (Timkey and van Schijndel, 2021) tend to be distributed unevenly across layers; see Figure 6 (top) for an example. Ablating an outlier direction can drastically harm performance (Kovaleva et al., 2021), so if LogitLens relies on the presence or absence of particular outlier dimensions, the perplexity of logit lens predictions might be spuriously high.

Even when controlling for rogue dimensions, we observe a strong tendency for the covariance matrices of hidden states at different layers to drift apart as the number of layers separating them increases (Figure 6, bottom). The covariance at the final layer often changes sharply relative to previous layers, suggesting the logit lens might "misinterpret" earlier representations.

One simple, general way to correct for drifting covariance is to introduce a learnable change of basis matrix A_ℓ, which learns to map from the output space of layer ℓ to the input space of the final layer. We have now arrived at the *tuned lens* formula, featuring a learned affine transformation for each layer:

$$\text{TunedLens}_\ell(\boldsymbol{h}_\ell) = \text{LogitLens}(A_\ell \boldsymbol{h}_\ell + \mathbf{b}_\ell) \quad (8)$$

We refer to $(A_\ell, \mathbf{b}_\ell)$ as the *translator* for layer ℓ.

Loss function. We train the translators to minimize KL between the tuned lens logits and the final layer logits:

$$\operatorname*{argmin} \mathop{\mathbb{E}}_{\boldsymbol{x}} \Big[D_{KL}(f_{>\ell}(\boldsymbol{h}_\ell) \,||\, \text{TunedLens}_k(\boldsymbol{h}_\ell)) \Big] \quad (9)$$

where $f_{>\ell}(\boldsymbol{h}_\ell)$ refers to the rest of the transformer after layer ℓ. This can be viewed as a distillation loss, using the final layer distribution as a soft label (Sanh et al., 2019). It ensures that the probes are not incentivized to learn extra information over and above what the model has learned, which can become a problem when training probes with ground truth labels (Hewitt and Liang, 2019).

Implementation details. When readily available, we train translators on a slice of the validation set used during pretraining, and use a separate slice for evaluation. Since BLOOM and GPT-2 do not have publicly available validation sets, we use the Pile validation set (Gao et al., 2020; Biderman et al., 2022). The OPT validation set is also not publicly available, but a member of the OPT team helped us train a tuned lens on the OPT validation set. Documents are concatenated and split into uniform chunks of length 2048.

We evaluate all models on a random sample of 16.4M tokens from their respective pretraining validation sets. We leave out the final transformer layer for GPT-2 (Radford et al., 2019), GPT-NeoX-20B (Black et al., 2022), OPT (Zhang et al., 2022), and Pythia (Biderman et al., 2023), and include it for GPT-Neo (Black et al., 2021). We evaluate BLOOM (Scao et al., 2022) under both conditions in Figure 4.

In our initial experiments, we used SGD with Nesterov momentum, with a linear learning rate decay schedule over

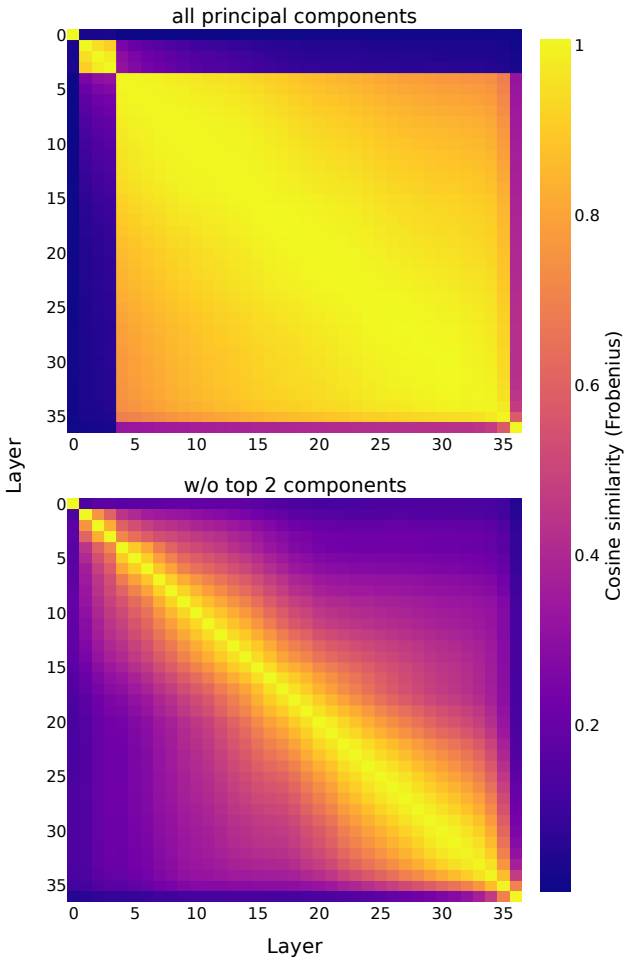

Figure 6. Pairwise similarities of hidden state covariance matrices across layers of Pythia 12B. Layer 4 introduces two outlier dimensions which dominate the covariance; removing them reveals smooth representational drift with depth. To control for varying hidden state norms, we measure the Frobenius cosine similarity, or $\frac{\langle A,B\rangle_F}{\|A\|_F\|B\|_F}$ for two matrices A and B.

250 training steps. We used a base learning rate of 1.0, or 0.25 when keeping the final transformer layer, and clip gradients to a norm of 1. We accumulated gradients as necessary to achieve a total batch size of 2^{18} tokens per optimizer step. We initialized all translators to the identity transform, and use a weight decay of 10^{-3}.

Muon optimizer. After the initial publication of this paper, Jordan et al. (2024) introduced the Muon optimizer. Using a fast orthogonalization algorithm, Muon attempts to maximize the effective rank (Roy and Vetterli, 2007) of the update applied to each matrix-valued parameter, thereby counteracting the well-known tendency of neural network gradients to be rank-deficient. Conceptually, Muon is the opposite of LoRA (Hu et al., 2022), a popular parameter-efficient finetuning method that constrains parameter updates to be low rank. Liu et al. (2025) find that Muon accelerates the training of modern large language models compared to AdamW.

Strikingly, we find that using Muon instead of SGD for training the tuned lens *dramatically* accelerates training, allowing us to achieve much lower KL divergence losses than were possible in our initial experiments. Essentially, all of the tuned lenses trained and evaluated in the earlier versions of this paper were severely undertrained. The weight matrices of Muon-trained lenses have Frobenius norms that are often several times larger than SGD-trained ones, showing that they are much further from the logit lens than our original lenses— they are much better "tuned," as it were. We opted not to re-do all our experiments with tuned lenses trained using Muon, but we encourage practitioners to use Muon in future experiments.

Results. We plot tuned lens perplexity as a function of depth for the Pythia models and GPT-NeoX-20B in Figure 5[3]; results for other model families can be found in Appendix A.

We find that the tuned lens resolves the problems with the logit lens discussed in Section 2: it has significantly lower bias (Figure 3), and much lower perplexity than the logit lens across the board (Figure 5, Appendix A).

Transferability across layers. We find that tuned lens translators can usually zero-shot transfer to nearby layers with only a modest increase in perplexity. Specifically, we define the *transfer penalty* from layer ℓ to ℓ' to be the expected increase in cross-entropy loss when evaluating the tuned lens translator trained for layer ℓ on layer ℓ'.

We report transfer penalties for the largest Pythia model in Figure 7. Overall, transfer penalties are quite low, especially for nearby layers (entries near the diagonal in Figure 7). Comparing to the two plots in Figure 6, we notice that transfer penalties are strongly negatively correlated with covariance similarity (Spearman $\rho = -0.78$). Unlike Figure 6, however, Figure 7 is not symmetric: transfer penalties are higher when training on a layer with the outlier dimensions (Layer 5 and later) and testing on a layer without them, than the reverse.

Transfer to fine-tuned models. We find that lenses trained on a base model transfer well to fine-tuned versions of that base model, with no additional training of the lens. Transferred lenses substantially outperform the Logit Lens and compare well with lenses trained specifically on the fine-tuned models. Here a transferred lens is one that makes use of the fine-tuned models unembedding, but simply copies

[3]Pythia and GPT-NeoX-20B were trained using the same architecture, data, and codebase (Andonian et al., 2021). While they're not officially the same model suite, they're more consistent than the OPT models.

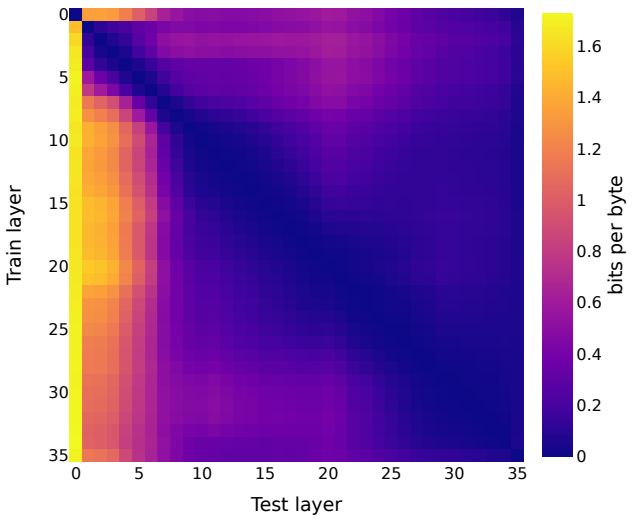

Figure 7. Transfer penalties for Pythia 12B. Each row corresponds to a single tuned lens probe trained on layer ℓ, and each column is a layer ℓ' on which probes are evaluated. Each cell shows the cross-entropy loss of probe ℓ evaluated on layer ℓ', *minus* its on-distribution loss (so that the diagonal entries are identically zero).

its affine translators from the lens trained on the base model.

As an example of the transferability of lenses to fine-tuned models we use Vicuna 13B (Chiang et al., 2023), an open source instruct fine-tuned chat model based on LLaMA 13B (Touvron et al., 2023). In Figure 13, Section A, we compare the performance of a lens specifically trained on Vicuna to a tuned lens trained only on LLaMA. Both Tuned Lenses were trained using a subsample of the RedPajama dataset (Together, 2023), an open source replication of LLaMA's training set. The performance of both lenses was then evaluated on the test set of Anthropic's Helpful Harmless (Bai et al., 2022) conversation dataset and the RedPajama Dataset. On the RedPajama Dataset we find, at worst, a 0.3 bits per byte increase in KL divergence to the models final output. On the helpful harmless corpus we find no significant difference between the transferred and trained lenses. These results show that fine-tuning minimally effects the representations used by the tuned lens. This opens applications of the tuned lens for monitoring changes in the representations of a module during fine-tuning and minimizes the need for practitioners to train lenses on new fine-tuned models.

Relation to model stitching. The tuned lens can be viewed as a way of "stitching" an intermediate layer directly onto the unembedding, with an affine transform in between to align the representations. The idea of model stitching was introduced by Lenc and Vedaldi (2015), who form a composite model out of two *frozen* pretrained models A and B, by connecting the bottom layers of A to the top layers of B. An affine transform suffices to stitch together independently trained models with minimal performance loss (Bansal et al., 2021; Csiszárik et al., 2021). The success of the tuned lens shows that model stitching works for different layers inside a single model as well.

Benefits over traditional probing. Unlike Alain and Bengio (2016), who train early exiting probes for image classifiers, we do not learn a new unembedding for each layer. This is important, since it allows us to shrink the size of each learned matrix from $|\mathcal{V}| \times d$ to $d \times d$, where $|\mathcal{V}|$ ranges from 50K (GPT-2, Pythia) to over 250K (BLOOM). We observe empirically that training a new unembedding matrix requires considerably more training steps and a larger batch size than training a translator, and often converges to a worse perplexity.

4. Measuring Causal Fidelity

Prior work has argued that interpretability hypotheses should be tested with causal experiments: an interpretation of a neural network should make predictions about what will happen when we intervene on its weights or activations (Olah et al., 2020; Chan et al., 2022). This is especially important for probing techniques, since it's known that probes can learn to rely on spurious features unrelated to the model's performance (Hewitt and Liang, 2019; Belinkov, 2022).

To explore whether the tuned lens finds causally relevant features, we will assess two desired properties:

1. Latent directions that are important to the tuned lens should also be important to the final layer output. Concretely, if the tuned lens *relies on* a feature[4] $\boldsymbol{v}$ in the residual stream (its output changes significantly when we manipulate $\boldsymbol{v}$) then the model output should also change a lot when we manipulate $\boldsymbol{v}$.

2. These latent directions should be important *in the same way* for both the tuned lens and the model. Concretely, if we manipulate the hidden state so that the tuned lens changes in a certain way (e.g. doubling the probability assigned to "dog") then the model output should change similarly. We will call this property *stimulus-response alignment*.

4.1. Causal basis extraction

To test Property 1, we first need to find the important directions for the tuned lens. Amnesic probing (Elazar et al.,

[4]For simplicity we assume the "features as directions" hypothesis (Elhage et al., 2022), which defines a "feature" to be the one-dimensional subspace spanned by a unit vector $\boldsymbol{v}$.